

Технический отчёт: лучшее ML-решение задачи «сборка пазла + реставрация изображения»

Кратко (TL;DR)

- **Строим модульный трёхстадийный пайплайн, а не end-to-end модель:** (1) пофрагментный *денойзер* для *матчинга*, дающий коррупция-инвариантные эмбединги фрагментов; (2) *решатель пазла*, расставляющий 576 фрагментов на основе обученной парной совместимости соседей + глобальной сборки; (3) полнокадровая сеть *реставрации/гармонизации* (NAFNet или Restormer, ширина 64), работающая на собранном изображении 480×480, чтобы убрать остаточные шум/размытие/JPEG и выровнять пофрагментные скачки яркости/контраста на швах. Оптимизируем комбинированный лосс **MS-SSIM + L1** и применяем ×8 геометрический self-ensemble на инференсе.
- **Используем два решающих преимущества, которые даёт условие задачи:** точная модель искажений задана (значит, можно синтезировать неограниченное число обучающих пар), и у нас есть чистые таргеты для трейна (значит, можно восстановить *точную* ground-truth перестановку для всех 7000 обучающих изображений через нормализацию освещённости + даунсэмплинг-матчинг признаков + венгерский алгоритм, получив полностью размеченные метки для решателя).
- **Реставрация — легко решаемая подзадача, и именно на ней набирается большая часть SSIM; сборка 576 фрагментов — настоящее узкое место.** Ни один опубликованный обучаемый решатель не достигает высокой точности на сетке 24×24, поэтому инженерные усилия вкладываем в устойчивый решатель по схеме «сначала денойз, потом матчинг», а финальную сеть реставрации делаем терпимой к небольшому числу неправильно расставленных фрагментов.

Ключевые выводы

- 1. Модульность лучше end-to-end.** Реставрация изолированных фрагментов 20×20 испытывает дефицит контекста; сборке пазла нужны коррупция-инвариантные признаки. Единая совместная сеть должна делать и то, и другое — её гораздо сложнее отлаживать в рамках 9-часового лимита сессии Kaggle. Модульный дизайн позволяет валидировать каждую стадию независимо и подставлять запасные варианты.
- 2. Известная модель искажений — подарок.** Поскольку пайплайн порчи полностью задан (яркость ±30, контраст 0.70–1.30, гауссов шум $\sigma=40\text{--}55$, гауссово размытие 3×3, JPEG quality 35–50), можно взять любой корпус чистых изображений (7000 таргетов плюс внешние наборы вроде DIV2K/Flickr2K) и сгенерировать неограниченный объём идеально спаренных данных (испорченное, чистое). Это превращает задачу реставрации из «слепой» (blind) в «с известной деградацией и учителем» — самый простой режим для современных реставраторов.

3. Ground-truth перестановки полностью восстановимы для трейна. Для каждой обучающей пары у нас есть и перемешанный-испорченный вход, и чистый таргет. Пофрагментная нормализация яркости/контраста (вычесть среднее, поделить на std) плюс сильное размытие+даунсэмплинг побеждает искажения, а матрица стоимостей 576×576 , решённая венгерским алгоритмом (`scipy.optimize.linear_sum_assignment`), даёт точную перестановку. Это даёт размеченные метки для обучения любого решателя и, что критично, для обучения классификатора парного соседства. Теоретическое обоснование матчинга на зашумлённых/уменьшенных патчах: при условии независимости шума от изображения, близость зашумлённых патчей влечёт близость их чистых версий, если патчи достаточно большие (Patch-Craft self-supervised denoising, arXiv:2211.09919). ZNCC/нормализованная кросс-корреляция инвариантна к аффинным изменениям освещённости (сдвиг + усиление), что точно соответствует искажению ± 30 по яркости / 0.7–1.3 по контрасту.

4. Современные реставраторы зрелы и напрямую применимы. NAFNet (Chen et al., «Simple Baselines for Image Restoration», ECCV 2022, arXiv:2204.04676) сообщает: «33.69 dB PSNR на GoPro (деблюринг), превосходя прежний SOTA на 0.38 dB при всего 8.4% его вычислительных затрат; 40.30 dB PSNR на SIDD (шумоподавление), превосходя прежний SOTA на 0.28 dB менее чем при половине его вычислительных затрат». Restormer (Zamir et al., CVPR 2022) — его трансформерный аналог; FBCNN (Jiang et al., ICCV 2021) специально создан для слепого удаления JPEG-артефактов. Победители челленджей NTIRE по шумоподавлению (2023/2025) стабильно комбинируют трансформер (Restormer) и CNN (NAFNet) с прогрессивным обучением, self-ensemble и ансамблированием моделей.

5. Обучаемые решатели пазлов рассыпаются задолго до 576 фрагментов. По Kim et al. (ESWA 2025): «FCViT с бэкбоном ViT-B/16 для 362 800 (9!) перестановок показывает наивысшие результаты по точности пазла и фрагмента — 78.9% и 90.6% — на задаче 3×3 на ImageNet», то есть громкие цифры относятся всего к 9 фрагментам. JPDVT (диффузионный vision transformer, arXiv:2404.07292) сообщает «68.7% успеха на ImageNet-1k и 71.3% на JPwLEG-3 для пазлов без пропущенных фрагментов» — снова на 3×3 ; при этом его собственная мотивация отмечает, что прежние дискриминативные методы «сталкиваются с ограничениями при решении пазлов с большим числом элементов». В бенчмарке испорченных пазлов простые трансформерные головы перестановки/координат резко деградируют с ростом размера пазла (доля идеальной реконструкции падает с ~91% при 6×6 до единиц процентов при 12×12), тогда как Positional Diffusion — наиболее устойчивая к масштабу обучаемая модель. Ни один опубликованный обучаемый решатель не демонстрирует высокой точности на 24×24 . Именно поэтому прагматичный выбор для теста — ручной решатель «денойз→матчинг совместимости», обученный на ваших восстановленных метках.

6. Пофрагментная порча ломает классический матчинг краёв. Mahalanobis Gradient Compatibility (MGC) Галлахера и L1-несовместимость границ Пайкина-Талья — две сильнейшие классические эвристики — быстро деградируют при пофрагментной порче цвета/яркости/контента. Dirauf et al. («Benchmarking Content-Based Puzzle Solvers on Corrupted Jigsaw Puzzles», arXiv:2507.07828, ICIAP 2025) обнаруживают, что «решатели, разработанные для

стандартных пазлов, резко теряют производительность при увеличении числа испорченных фрагментов... Среди эвристик решатель Галлахера показывает себя хуже всех, за ним Пайкин и Таль, а эвристика Yu et al. достигает наивысшей точности». Их ключевой вывод о смягчении: «модели глубокого обучения могут значительно повысить устойчивость через дообучение на аугментированных данных. В частности, продвинутая модель Positional Diffusion адаптируется особенно хорошо». Практический вывод — денойзить и нормализовать освещённость каждого фрагмента *перед* матчингом и/или обучать коррупция-инвариантную совместимость на ваших восстановленных метках.

Подробности

А. Структура задачи и откуда берётся SSIM

Финальная метрика — RGB SSIM (`skimage.metrics.structural_similarity`, `channel_axis=2`, `data_range=255`, дефолтный `win_size=7`, равномерное окно 7×7 , среднее по тесту). SSIM считается скользящим окном 7×7 по каждому каналу; оценка раскладывается на компоненты яркости, контраста и структуры. Важны два следствия:

- **SSIM нечувствителен к равномерным смещениям** (глобальный сдвиг яркости/цвета почти не двигает SSIM), но **очень чувствителен к малым пространственным сдвигам** — сдвинутый или неправильно поставленный блок 20×20 даёт большой локальный штраф SSIM по всем окнам 7×7 , пересекающим его швы. Значит, правильная расстановка стоит дорого, а остаточные межфрагментные швы яркости стоят относительно мало по сравнению со структурной ошибкой размещения.
- **Эксплоит «однотонной картинки»:** поскольку стабилизирующие константы SSIM (C_1 , C_2) доминируют в областях с низкой дисперсией, плоское серое изображение может получать обманчиво высокую оценку относительно своей перцептивной бесполезности. Команда явно хочет настоящее качество, поэтому мы оптимизируем реальную реставрацию (MS-SSIM+L1), а не играем на константе. Эксплоит упоминаем только как предостережение: пере-денойзенный, низкоконтрастный выход может набрать *больше* SSIM, чем более резкий, — поэтому тщательно валидируйте.

Точно воспроизведите скорер для локальной валидации. Вызывайте

`structural_similarity(pred, target, channel_axis=2, data_range=255, win_size=7)` на uint8 RGB-массивах, все остальные параметры — по умолчанию (то есть `gaussian_weights=False`, используется равномерный box-фильтр 7×7 и нормировка по выборочной ковариации). НЕ ставьте `gaussian_weights=True` — это меняет результат и рассинхронизирует с лидербордом. Усредняйте по валидационной выборке.

В. Восстановление ground-truth перестановок (только трейн)

Для каждой из 7000 обучающих пар:

1. Нарезьте вход и таргет на сетку 24×24 из фрагментов 20×20 (по 576 в каждом).

2. Для каждого фрагмента вычислите **устойчивый к порче дескриптор**: пофрагментная нормализация (вычесть среднее, поделить на std по каналу, чтобы победить сдвиг ± 30 по яркости / 0.7–1.3 по контрасту), сильное гауссово размытие и даунсэмплинг до $\sim 4\text{--}6$ px (чтобы подавить шум $\sigma=40\text{--}55$ и JPEG-блоки; матчинг на агрессивно низкочастотных представлениях восстанавливает близость чистых патчей, так как независимый шум усредняется на больших патчах). Опционально добавьте небольшую гистограмму хром в LAB как тай-брейкер (хрома меньше страдает от яркости).
3. Постройте матрицу стоимостей 576×576 из расстояний между дескрипторами входных фрагментов и ячеек таргета (используйте расстояние ZNCC/нормализованной корреляции — оно инвариантно к аффинным изменениям освещённости, то есть равно к здешней порче яркости+контраста). Опционально добавьте косинусное расстояние по признакам среднего уровня из ImageNet-предобученного ResNet (layer2/3), что эмпирически устойчивее, чем сырой цвет, для матчинга патчей с разной внешностью (Hungarian-on-CNN-features matching, arXiv:1904.10348, использовал признаки conv3 AlexNet с косинусной близостью ровно для такого рода назначения source→target).
4. Решите через `scipy.optimize.linear_sum_assignment` для доказуемо оптимального взаимно-однозначного назначения → точная метка перестановки.
5. **Проверьте** каждую восстановленную перестановку: соберите ячейки таргета в восстановленном порядке и подтвердите почти идеальную реконструкцию; пометьте/почините любую пару с аномально высокой стоимостью назначения. Ожидается практически 100% восстановление при наличии чистых таргетов.

Эти метки питают обучение с учителем компонентов решателя ниже.

С. Сборка пазла на тесте (таргета нет)

Это самая сложная часть. Рассмотрены три семейства:

- **(а) Обучаемые глобальные решатели** (регрессия координат FCViT, Positional Diffusion, диффузионный трансформер JPDVT, трансформерные головы перестановки с Gumbel-Sinkhorn). Gumbel-Sinkhorn (Mena et al., ICLR 2018, arXiv:1802.08665) релаксирует недифференцируемую матрицу перестановки в дважды стохастическую через оператор Синкхорна, делая обучение end-to-end. В принципе привлекательно, но все эти методы рассыпаются задолго до 576 фрагментов на опубликованных бенчмарках (см. Ключевой вывод 5). Не годятся как единственный решатель на 24×24 .
- **(б) Классические эвристики матчинга краёв** (MGC + жадная/древовидная сборка, L1 Пайкина-Талья, ЛП Yu et al.). Сильны на чистых пазлах — ЛП Yu et al. самая устойчивая классическая эвристика — но все деградируют при пофрагментной порче.
- **(в) Рекомендуемый гибрид: денойз → обучаемая парная совместимость → глобальная сборка.** Самый устойчивый путь, напрямую использующий восстановленные метки.

Рекомендуемая конструкция решателя:

1. **Денойзер для матчинга:** небольшая CNN-реставратор, запускаемая пофрагментно только чтобы сделать матчинг устойчивым — она не обязана быть идеальной, лишь достаточно раскрыть настоящие края и нормализовать внешность.
2. **Обучаемая сеть парного соседства:** компактная CNN (сиамская/по краевым полоскам, 6–8 слоёв), берущая примыкающие краевые полосы двух фрагментов (например, последние 3–5 пиксельных столбцов левого фрагмента и первые 3–5 правого) и выдающая $P(\text{соседи})$ для каждого из 4 отношений (право/лево/верх/низ). Обучайте на испорченных фрагментах с метками соседства из восстановленных перестановок (позитивы = реально соседние; негативы = случайные несоседние, с *hard-negative mining*). Это учит *коррупция-инвариантной* совместимости, недоступной классическим $MGC/L1$. Как альтернатива — учите пофрагментные краевые эмбединги так, чтобы у пространственно-соседних фрагментов эмбединги были совместимы (контрастивная задача).
3. **Глобальная сборка:** переведите парные оценки в расстановку. Два варианта: (i) постройте тензор совместимости $576 \times 576 \times 4$ и запустите жадную древовидную/петле-ограниченную расстановку в стиле Галлахера, стартуя с наиболее уверенных пар; (ii) для более глобального оптимума сформулируйте как задачу назначения/линейного программирования по известным позициям сетки 24×24 . Поскольку размеры сетки известны (ровно 24×24), можно также закрепить границу и заполнять внутрь. Используйте «лучших приятелей» (*best-buddies* — взаимно-наилучшие пары соседей), чтобы стартовать и обнаруживать/чинить ошибки.
4. **Устойчивость к ошибкам расстановки:** SSIM штрафует неправильные блоки локально, но не глобально, так что несколько ошибок среди 576 переживаемы. Добавьте проверку согласованности — пересчитайте согласие каждого поставленного фрагмента с его 4 соседями и для низкосогласованных фрагментов попробуйте проход *swap-repair* (обмены).

Реставрация до или после сборки? Делайте *лёгкий* денойз до матчинга (чтобы решатель заработал), но *тяжёлую* реставрацию после сборки (на полном изображении 480×480 , где у сети полный контекст и она может убрать швы). Совместное *end-to-end* решение сборки+реставрации не рекомендуется: его сложно обучать, сложно отлаживать, а дискретная перестановка делает градиенты неудобными даже с Gumbel-Sinkhorn на этом масштабе.

D. Реставрация изображения (движок SSIM)

Выбор архитектуры. Два проверенных бэкбона:

- **NAFNet (рекомендуемый основной):** U-образный одностадийный энкодер-декодер из NAFBlock (LayerNorm, depthwise conv, SimpleGate = split-channels-and-multiply вместо нелинейной активации, Simplified Channel Attention). SOTA-качество при малых вычислениях — идеально для Kaggle. Используйте ширину 64 с блоками энкодера [2,2,4,8],

middle 12, декодера [2,2,2,2] (конфиг шумоподавления SIDD) или конфиг деблюринга [1,1,1,28]/1/[1,1,1,1]. Ширина 32 — быстрый запасной вариант.

- **Restormer (рекомендуемый вторичный для ансамбля):** эффективный трансформер с multi-Dconv head transposed attention (поканальный attention, линейный по пространственному разрешению) и gated-Dconv feed-forward; 4-уровневый энкодер-декодер с блоками [4,6,6,8]. Ловит дальнюю структуру; дополняет локальные сильные стороны NAFNet. Победители NTIRE сливают именно эту пару трансформер+CNN.

Обучающие данные. Обучайте на смеси (i) 7000 предоставленных пар и (ii) синтетически испорченных внешних чистых изображений (DIV2K, Flickr2K плюс сами 7000 чистых таргетов). Генератор синтетики должен воспроизводить порчу *пофрагментно на 20×20* для домена «собранного испорченного изображения», а также может применяться глобально для предобучения. Обучение и на реальных предоставленных, и на синтетических парах строго лучше любого по отдельности: синтетика даёт объём и точное покрытие деградации; предоставленные пары гарантируют отсутствие доменного разрыва.

Две модели реставрации, две роли.

- *Денойзер для матчинга* (маленький NAFNet ширины 16–32, обучен на одиночных фрагментах 20×20): быстрый, нужен лишь чтобы помочь решателю.
- *Полнокадровый реставратор + гармонизатор* (NAFNet ширины 64 и/или Restormer, обучен на собранных парах 480×480 испорченное→чистое): это ключевая модель. Критично: формируйте её обучающие входы, собирая *пофрагментно-испорченные* фрагменты в правильную сетку (по ground-truth перестановке), чтобы сеть училась убирать ровно те межфрагментные швы яркости/контраста и границы блоков, которые она увидит на тесте. Эта единая сеть одновременно делает денойз, деблюринг, удаление JPEG и межфрагментную гармонизацию — реставратор «всё-в-одном».

Обработка пофрагментных швов яркости/контраста. Поскольку у каждого фрагмента своя яркость (± 30) и контраст (0.7–1.3), даже идеально расставленное изображение показывает видимую шахматку скачков интенсивности. Три взаимодополняющих средства, в порядке рекомендации: (1) обучить полнокадровый реставратор на входах, собранных из испорченных фрагментов, чтобы он учился гармонизировать швы end-to-end (проще всего, устойчивее всего); (2) добавить явную голову пофрагментной аффинной коррекции, которая предсказывает и инвертирует (усиление, сдвиг) каждого фрагмента перед реставратором; (3) финальный лёгкий проход глобальной гармонизации (например, патч-нормализация/color-transfer, как в сетях гармонизации изображений). Обычно достаточно варианта (1), и он избегает хрупкой пофрагментной оценки.

Функция потерь. Метрика — SSIM, поэтому оптимизируйте взвешенный **MS-SSIM + L1** лосс. По Zhao et al. («Loss Functions for Image Restoration with Neural Networks», IEEE Trans. Computational Imaging 3(1):47–57, 2017, arXiv:1511.08861), рекомендуемая смесь:
$$\text{Mix} = \alpha \cdot \text{MS-SSIM_loss} + (1 - \alpha) \cdot (\text{L1, взвешенный гауссианой})$$
 при « $\alpha=0.84$... выбрано так, чтобы вклад двух

лоссов был примерно сбалансирован», а масштабные веса MS-SSIM $\omega = \{0.5, 1, 2, 4, 8\}$.

Обоснование из той же работы: «MS-SSIM сохраняет контраст в высокочастотных областях... ℓ_1 сохраняет цвета и яркость», и «сеть, комбинирующая MS-SSIM и ℓ_1 , достигает лучших результатов по всем рассмотренным метрикам качества». Чистый L1/L2 недобирает по SSIM; чистый SSIM-лосс уводит цвета (SSIM «не особо чувствителен к равномерным смещениям», вызывая сдвиги яркости/цвета). Опционально добавьте небольшой перцептивный член VGG/LPIPS для текстур, но держите его малым — перевес перцептивных/GAN-лоссов может ухудшить SSIM. Практическое расписание: сначала обучайте на L1 для стабильности, затем дообучайте на MS-SSIM+L1.

Оптимизатор/расписание. AdamW (betas 0.9/0.9), стартовый LR $\sim 1e-3$ (NAFNet, по его официальным конфигам) или $\sim 3e-4$ (Restormer), косинусный отжиг до $\sim 1e-7$, 100k–400k итераций в зависимости от бюджета времени. Прогрессивное обучение (рост размера патча, например 128→192→256) улучшает полнокадровую производительность, как у победителей NTIRE. Используйте mixed precision (AMP) и EMA весов.

Test-time augmentation. Применяйте $\times 8$ геометрический self-ensemble (4 поворота $\times 2$ отражения): преобразуйте вход, реставрируйте, обратно преобразуйте, усредните (Lim et al., EDSR, arXiv:1707.02921). Даёт небольшой, но стабильный прирост SSIM бесплатно (без дообучения). Ансамблируйте выходы NAFNet + Restormer (простое усреднение или обученные веса) для дополнительной прибавки, ровно как в NTIRE-победных пайплайнах шумоподавления. Если сеть использует глобальные операции (пулинг/нормализацию), примените TLC/TLSC (test-time local converter), чтобы избежать несоответствия «обучение на патчах / тест на полном изображении» (ECCV 2022).

Е. Стратегия валидации

Отложите ~ 700 из 7000 обучающих пар (совпадает с размером теста) как локальную валидацию, стратифицированную под тестовое распределение. Поскольку вы можете восстанавливать перестановки и знаете порчу, можно измерять каждую стадию отдельно: (i) точность расстановки решателя (% верно поставленных фрагментов), (ii) SSIM реставрации при *оракульной* расстановке, (iii) end-to-end SSIM с вашим реальным решателем. Это разложение точно говорит, где теряется SSIM. Всегда оценивайте точным skimage-вызовом выше. Отслеживайте разрыв между SSIM при оракульной расстановке и при реальной, чтобы решить, во что вкладываться больше — в решатель или в реставратор.

Ф. Инженерия под Kaggle

- **Загрузка данных:** нарежьте сетки на лету; кэшируйте декодированные PNG; используйте `num_workers` ≥ 2 . Для синтетики портите чистые изображения пофрагментно прямо в DataLoader.
- **Память/время:** NAFNet ширины 64 на патчах 256×256 комфортно влезает в P100 16GB / T4 с AMP; батч 8–16. Полнокадровый инференс 480×480 дешёвый. Планируйте обучение

под 9-часовые сессии; чекпойнтите каждые N итераций и возобновляйтесь между сессиями. Парная сеть решателя крошечная и быстрая.

- **Инференс/сабмишн:** для каждого тестового изображения → нарезка → денойз для матчинга → решатель → сборка испорченного 480×480 → полнокадровая реставрация (+ТТА, +ансамбль) → клип в [0,255] uint8 → сохранить `img_XXXXXX.png` (RGB) → упаковать 700 файлов в `submission.zip`. Убедитесь, что имена файлов точно совпадают с тестовыми.

Рекомендации

Стройте по этапам, чтобы гарантированно иметь сдаваемый пайплайн рано, затем улучшайте:

1. **Веха 1 — скорер + бейзлайн (день 1):** реализуйте точный SSIM-скорер и тривиальный бейзлайн (вернуть испорченный-перемешанный вход или его глобально денойзенную версию). Устанавливает водопровод сабмишна и нижний порог. *Триггер к смене плана:* если даже испорченный вход набирает неожиданно много, проверьте поведение метрики перед вложениями.
2. **Веха 2 — восстановление перестановок + оракульная реставрация (день 1-2):** восстановите все обучающие перестановки (венгерский алгоритм). Обучите полнокадровый NAFNet ширины 64 на *оракульно собранных* парах испорченное→чистое на L1, затем MS-SSIM+L1. Измерьте SSIM при оракульной расстановке — это ваша верхняя граница. *Порог:* если оракульный SSIM уже высок (например, >0.9), решатель — это вся игра; приоритезируйте его.
3. **Веха 3 — решатель (день 2-3):** обучите денойзер для матчинга и сеть парного соседства на восстановленных метках; реализуйте сборку greedy/best-buddies с проходом починки. Измерьте точность расстановки и end-to-end SSIM.
4. **Веха 4 — полировка (день 3+):** добавьте Restormer, ансамбль моделей + ×8 self-ensemble, прогрессивное обучение, пофрагментную аффинную коррекцию если швы остаются, и swap-repair для ошибок решателя.

Запасные варианты на каждой стадии: реставрация → NAFNet ширины 32 или простой DnCNN/U-Net при нехватке вычислений, либо предобученный FBCNN для JPEG-компоненты; решатель → если обученная совместимость недобирает, откат на денойз→MGC greedy (Галлахер) или ЛП Yu et al. (самая устойчивая классическая эвристика при порче); лосс → чистый L1 если обучение с MS-SSIM нестабильно; гармонизация → полагаться только на полнокадровый реставратор, если явная аффинная оценка шумная.

Куда потратить последний час вычислений: ансамблирование и ×8 self-ensemble на модели реставрации дают самый надёжный маржинальный SSIM; проходы починки решателя помогают только если узкое место — точность расстановки (проверьте оракульный разрыв).

Оговорки

- **Сборка 576 фрагментов действительно не решена с высокой точностью в литературе.** Опубликованные цифры обучаемых решателей — на крошечных числах фрагментов (FCViT/JPDVT на 3×3), а классические эвристики рассыпаются при пофрагментной порче. Ожидайте, что сборка будет доминирующим источником потери SSIM; проектируйте реставратор устойчивым к нескольким неправильным блокам, а не в расчёте на идеальную расстановку.
- **SSIM может вознаграждать пере-сглаживание.** Валидируйте, что ваша MS-SSIM+L1 модель не выигрывает за счёт размытия; сравните с более резким вариантом на точном скорере.
- **Соответствие домена важно:** обучайте полнокадровый реставратор на входах, собранных из пофрагментно-испорченных фрагментов (а не глобально-испорченных изображений), иначе он не научится убирать конкретные межфрагментные швы.
- **Крайние случаи восстановления перестановок:** почти однородные фрагменты (небо, плоские области) могут матчиться неправильно; глобальный венгерский решатель это смягчает, но проверяйте реконструкции и помечайте назначения с высокой стоимостью.
- **Некоторые заявленные цифры решателей — на других данных/числах фрагментов** (FCViT на $3\times 3 = 78.9\%/90.6\%$; JPDVT на $3\times 3 = 68.7\%/71.3\%$). Конкретную цифру «45% идеальной реконструкции при 150 фрагментах», которую я изначально приводил для JPDVT, не удалось подтвердить по источнику — она удалена; трактуйте все межбенчмарковые цифры как ориентировочные, не как гарантии на 24×24 .
- **Порядок операций порчи** (контраст→яркость→шум→размытие→JPEG или иной) предполагается; валидируйте выходы синтетического генератора против сигнатуры шума/JPEG предоставленных train-входов и трактуйте порядок как небольшой гиперпараметр при неоднозначности.

СПЕЦИФИКАЦИЯ РЕАЛИЗАЦИИ ДЛЯ КОДОВОГО АГЕНТА

Постройте следующую структуру модулей/файлов на PyTorch. Не пишите монолитный код; держите каждый модуль независимо тестируемым.

Структура файлов

```
project/
  config.py          # все гиперпараметры, пути, сиды
  data/
    degradation.py   # точный симулятор порчи из условия
    grid.py          # нарезка/сборка 480x480 <-> сетка 24x24 из 20x20
    permutation_recovery.py # восстановление GT-перестановки венгерским алгоритмом
    datasets.py      # torch Datasets для каждой обучающей задачи
```

```

models/
  nafnet.py           # NAFNet (импорт/адаптация из megvii-research/NAFNet)
  restormer.py        # Restormer (импорт из официального репо)
  matching_denoiser.py # маленький NAFNet ширины 16-32 для денойза фрагментов
  compatibility_net.py # CNN парного соседства
solver/
  assemble.py         # сборка greedy/best-buddies + починка
train/
  train_restorer.py
  train_matching_denoiser.py
  train_compatibility.py
infer/
  solve_and_restore.py # end-to-end инференс на тесте
  make_submission.py   # упаковка submission.zip
eval/
  ssim.py              # обёртка точного skimage-скорера
  validate.py

```

Спецификации модулей

data/degradation.py — симулятор порчи. Функция `corrupt_fragment(frag_uint8) -> uint8`, применяющая в этом порядке к каждому фрагменту 20×20 : (1) контраст: `out = 128 + contrast * (x - 128)`, $\text{contrast} \sim U(0.70, 1.30)$; (2) яркость: `out += U(-30, 30)`; клип $[0, 255]$; (3) аддитивный гауссов шум $\sigma \sim U(40, 55)$ (в единицах 0–255); (4) гауссово размытие 3×3 ; (5) перекодирование JPEG на $\text{quality} \sim U(35, 50)$ (используйте PIL/cv2 encode-decode). Сэмплируйте все параметры независимо для каждого фрагмента. Предоставьте `corrupt_image(clean_480) -> degraded_480`, которая нарезает на 576 фрагментов, портит каждый независимо и пересобирает. Точно соблюдайте порядок/клиппинг; unit-тестируйте сравнением статистик с предоставленными train-входами. При неоднозначности порядка операций трактуйте порядок как небольшой гиперпараметр и выберите вариант, чьи выходы лучше всего совпадают с сигнатурой шума/JPEG предоставленных входов.

data/grid.py. `to_fragments(img_480) -> (576, 20, 20, 3)` и `from_fragments(frag, perm) -> img_480`. `perm[i]` = позиция i -го входного фрагмента в сетке. Включите хелперы $\text{row/col} \leftrightarrow \text{index}$ для сетки 24×24 .

data/permutation_recovery.py. `recover_permutation(input_480, target_480) -> perm`: нарежьте оба; постройте дескрипторы (поканальная пофрагментная нормализация $\text{mean/std} \rightarrow$ гауссово размытие $\rightarrow$ даунсэмплинг до $\sim 5 \times 5 \rightarrow \text{flatten}$; опционально добавьте признаки среднего уровня ResNet и гистограмму хром LAB); вычислите стоимость 576×576 через `1 - ZNCC` (или L2 на нормализованных дескрипторах); `scipy.optimize.linear_sum_assignment`; верните `perm` и стоимости назначений. Предоставьте `verify(perm, input, target)`, которая реконструирует и отчитывается о согласии; помечайте изображения с высокой остаточной стоимостью.

`models/nafnet.py`. Импортируйте официальный блок/сеть NAFNet (github.com/megvii-research/NAFNet). Предоставьте конфиги: `NAFNet_w64_sidd` (ширина 64, enc [2,2,4,8], middle 12, dec [2,2,2,2]) для полнокадрового реставратора; `NAFNet_w32` — запасной; `NAFNet_w16` — для денойзера матчинга. Поддержите residual-обучение (предсказывать $\text{clean} = \text{input} + \text{net}(\text{input})$).

`models/restormer.py`. Импортируйте официальный Restormer (4 уровня, блоки [4,6,6,8], MDTA+GDFN). Используется как партнёр по ансамблю и обучается так же, как реставратор.

`models/compatibility_net.py`. Вход: две примыкающие краевые полосы (каждая $3-5 \text{ px} \times 20 \text{ px} \times 3$, от двух фрагментов вдоль общей границы), конкатенированные. Тело: небольшой conv-стек (Conv-BN-GELU $\times 4$) $\rightarrow$ global pool $\rightarrow$ MLP $\rightarrow$ 1 логит на отношение. Предоставьте 4 головы отношений (или подавайте отношение как вход и шарьте веса). Выход: $P(\text{соседи})$. Обучайте с BCE на позитивных (реально соседних) против негативных (случайных) пар из восстановленных перестановок, с hard-negative mining высоко-оценённых несоседей. Опционально реализуйте вариант с эмбедингами: пофрагментный краевой энкодер, дающий эмбединги right/left/top/bottom, с контрастивным лоссом, чтобы соседние края были близки в пространстве эмбедингов.

`solver/assemble.py`. `solve(fragments) -> perm`: (1) денойзить фрагменты для матчинга; (2) вычислить тензор парной совместимости $576 \times 576 \times 4$ через `compatibility_net`; (3) найти best-buddies; (4) жадная расстановка на сетке 24×24 со старта с самой уверенной best-buddy пары, расширяясь к пустым соседям по максимальной совместимости (в стиле Галлахера), уважая фиксированные границы 24×24 ; (5) проход починки: для каждого поставленного фрагмента вычислить среднее согласие с 4 соседями; для наименее согласованных фрагментов попробовать парные обмены, повышающие суммарную совместимость. Предоставьте запасной `solve_mgc()`, реализующий денойз $\rightarrow$ MGC $\rightarrow$ greedy на случай, если обученная сеть недобирает.

`train/train_restorer.py`. Датасет: собирайте испорченное 480×480 из пофрагментно-испорченных фрагментов, размещённых по ground-truth перестановке $\rightarrow$ таргет = чистое 480×480 . Также подмешивайте глобально-синтетические испорченные пары из внешних чистых изображений. Кропайте патчи 256×256 (прогрессивно $128 \rightarrow 192 \rightarrow 256$). Лосс: разогрев на L1, затем $0.84 \cdot (1 - \text{MS-SSIM}) + 0.16 \cdot (\text{L1, взвешенный гауссианой})$, масштабные веса MS-SSIM {0.5, 1, 2, 4, 8}. Оптимизатор AdamW (betas 0.9/0.9), LR $1e-3$ (Restormer $3e-4$) косинус $\rightarrow 1e-7$, AMP, батч 8-16, 100k-400k итераций, чекпойнт каждые 5k, ЕМА весов. Валидируйте точным SSIM на отложенных 700.

`train/train_matching_denoiser.py`. Датасет: одиночные фрагменты 20×20 , испорченное $\rightarrow$ чистое. Маленький NAFNet, L1-лосс, быстро (десятки тысяч итераций). Цель — устойчивость матчинга, не идеальность.

`train/train_compatibility.py`. Постройте датасет соседства из восстановленных перестановок по всем 7000 обучающих изображений (миллионы пар краёв). BCE-лосс, AdamW, сбалансированный по классам сэмплинг с hard-negative mining. Валидируйте на отложенных изображениях по точности расстановки при подключении в `assemble.solve`.

`infer/solve_and_restore.py`. Для каждого тестового изображения: нарезка → денойз для матчинга → `assemble.solve` → `from_fragments` для сборки испорченного 480×480 → полнокадровая реставрация NAFNet (+ Restormer), каждая с ×8 self-ensemble, усреднить выходы → клип → uint8.

`infer/make_submission.py`. Прогоните инференс по всем 700 тестовым изображениям, сохраните `img_XXXXXX.png` в соответствии с именами входов, упакуйте в `submission.zip`. Проверьте: 700 файлов, RGB, 480×480.

`eval/ssim.py`. Оберните `skimage.metrics.structural_similarity(pred, target, channel_axis=2, data_range=255, win_size=7)` со всеми остальными параметрами по умолчанию; работайте на uint8. `mean_ssim(preds, targets)` возвращает среднее в стиле лидерборда.

Порядок реализации и вехи

1. `eval/ssim.py` + `data/grid.py` + `infer/make_submission.py` с тривиальным бейзлайном → подтвердить, что end-to-end сабмишн работает.
2. `data/degradation.py` (валидация против предоставленных входов) + `data/permutation_recovery.py` → восстановить все перестановки, проверить реконструкции.
3. `train/train_restorer.py` на оракульно-собранных входах → измерить SSIM при оракульной расстановке (верхняя граница).
4. `models/compatibility_net.py` + `train/train_compatibility.py` + `solver/assemble.py` → измерить точность расстановки + end-to-end SSIM.
5. Добавить Restormer, ансамблирование, ×8 ТТА, пофрагментную аффинную коррекцию, починку решателя.

Запасные варианты

- Реставратор слишком медленный → NAFNet ширины 32; или предобученный проход FBCNN для JPEG плюс лёгкий денойзер.
- Решатель недобирает → денойз → MGC greedy (Галлахер) или ЛП Yu et al.; закрепите границы, используя известные размеры 24×24.
- Обучение с MS-SSIM нестабильно → только L1, добавить MS-SSIM на дообучении.
- Швы остаются → добавить явную голову пофрагментной коррекции (усиление, сдвиг) перед реставратором.

Ключевые ссылки (архитектуры, репозитории, статьи)

- NAFNet — Chen et al., «Simple Baselines for Image Restoration», ECCV 2022 (arXiv:2204.04676); `github.com/megvii-research/NAFNet` (предобученные веса SIDD/GoPro, конфиги). Также победил в NTIRE 2022 Stereo Image SR (NAFSSR).

- Restormer — Zamir et al., CVPR 2022 (arXiv:2111.09881); официальное репо с предобученными весами denoise/deblur.
- FBCNN (слепое удаление JPEG-артефактов) — Jiang et al., ICCV 2021 (arXiv:2109.14573); github.com/jiaxi-jiang/FBCNN.
- Функции потерь — Zhao et al., «Loss Functions for Image Restoration with Neural Networks», IEEE TCI 2017 (arXiv:1511.08861) — рецепт MS-SSIM+L1, $\alpha=0.84$.
- Дифференцируемые перестановки Gumbel-Sinkhorn — Mena et al., ICLR 2018 (arXiv:1802.08665); github.com/google/gumbel_sinkhorn.
- Решатели пазлов — Gallagher MGC (CVPR 2012); Paikin & Tal (CVPR 2015); Yu et al. LP (BMVC 2016); FCViT (Kim et al., ESWA 2025); Positional Diffusion (Giuliani et al., Pattern Recognition Letters 2024); JPDVT (arXiv:2404.07292); бенчмарк испорченных пазлов, Dirauf et al. (arXiv:2507.07828, ICIAP 2025).
- Устойчивость матчинга патчей — Patch-Craft self-supervised denoising (arXiv:2211.09919); Hungarian-on-CNN-features matching (arXiv:1904.10348).
- $\times 8$ self-ensemble — Lim et al., EDSR (arXiv:1707.02921); согласованность train-test TLC/TLSC (ECCV 2022).